



OS FINAL PROJECT REPORT

Process Scheduling – Round Robin, Preemptive Scheduling : SRT

Course Code	: SECR2043
Course Name	: Operating System
Session-Semester	: 2024/2025-2

Prepared by : Muhammad Naim Bin Abdullah (A22EC0121)
Muhammad Mukhriz Al Iman Bin (A22EC0042)
Mohd Raffi
Mikhail Bin Yassin (A21EC0053)
Akram Ali Murshed (A23CS4022)

Group : AyamGepuk

Section : 02

Lecturer : Dr. MUHAMMAD ZAFRAN BIN MUHAMMAD ZALY
SHAH

Date : 5 / 7 / 2025

Presentation :

Video

--

Table of Contents

Traffic Light Timing Control using Round Robin Scheduling.....	1
Abstract.....	1
1.0 Introduction.....	1
2.0 Problem Statement.....	1
3.0 Related Works.....	1
4.0 Methodology.....	2
4.1 Project Management Flowchart.....	2
4.2 Hardware and Software Requirements.....	2
4.3 Task Distribution.....	2
5.0 Design & Implementation.....	3
5.1 System Flowchart.....	3
5.2 Code Implementation.....	3
6.0 Results & Discussion.....	6
6.1 Interface and Execution.....	6
6.2 Output Analysis.....	6
6.3 Discussion.....	7
7.0 Future Work & Conclusion.....	8
7.1 Future Work.....	8
7.2 Conclusion.....	8

Traffic Light Timing Control using Round Robin and SRT Scheduling

Abstract

Urban traffic congestion contributes to longer travel times, higher fuel consumption, and increased environmental pollution. This project models traffic light timing control by applying the Round Robin scheduling algorithm, which allocates green-light intervals evenly across all intersection approaches. Through this simulation, we illustrate how an OS-inspired scheduler can be leveraged to streamline vehicular flow and shorten wait times, offering insights for urban planners and traffic engineers seeking data-driven solutions to congestion.

1. Introduction

Urban areas worldwide struggle with traffic backups that not only delay commuters but also drive up fuel expenses and escalate emissions. One effective mitigation strategy lies in optimizing traffic signal timing. In this study, we adopt the Round Robin scheduling algorithm—commonly used for process management in operating systems—to govern the sequence and duration of green lights at a four-way intersection. By treating each direction as a “process” that receives a fixed time slice, our simulation highlights the potential of systematic, time-based control to improve overall traffic throughput and reduce idle waiting.

2. Problem Statement

Animated traffic simulations enable researchers and engineers to explore and refine signal-control strategies in a controlled environment. The primary goal of this project is to showcase how Round Robin scheduling can be implemented to balance green-light allocation among multiple traffic streams, thereby enhancing flow and cutting down congestion at intersections. We will develop a Python-based simulator, analyze performance metrics such as average wait time, and assess the algorithm’s effectiveness. This tool is tailored for traffic engineers, urban planners, and scholars focused on intelligent transportation systems.

3. Related Works

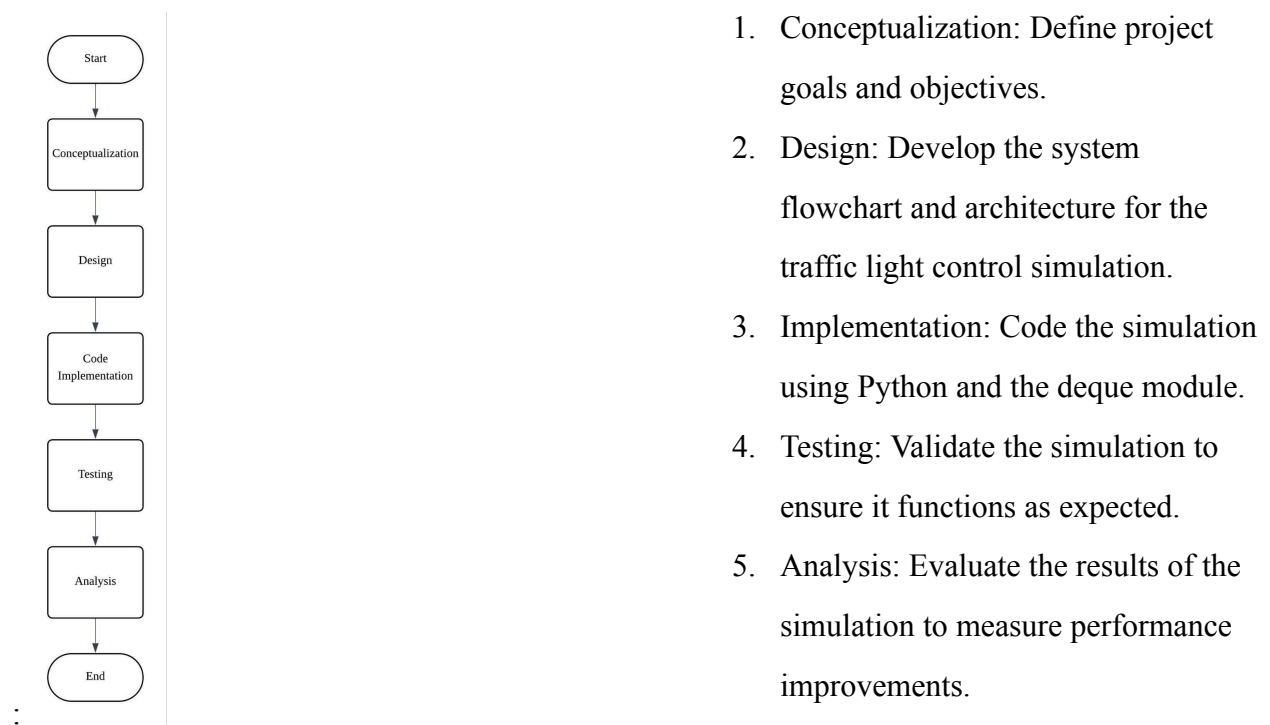
Round Robin scheduling is a well-known concept in operating systems for process management and has been adapted to traffic light control systems to enhance intersection efficiency. Other related animation projects in traffic simulations have explored various approaches to optimize traffic flow using different scheduling algorithms. This project builds on these foundations by applying Round Robin scheduling to a simulated traffic environment.

4. Methodology

4.1 Project Management Flowchart

The Project Management Flowchart visually outlines the sequence of activities and phases involved in the project. It includes steps such as conceptualization, design, implementation, testing, and analysis.

Steps in the Flowchart



4.2 Hardware and Software Requirements

- Software: Python for coding, deque module for algorithm scheduling.
- Hardware: Standard computational resources for running simulations.

4.3 Task Distribution

- Literature Review: MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI
- Simulation Design and Coding: MUHAMMAD NAIM BIN ABDULLAH
- Testing and Analysis: MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI & MUHAMMAD NAIM BIN ABDULLAH
- Report Writing: MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI & MUHAMMAD NAIM BIN ABDULLAH

5. Design & Implementation

```
import time
import os
from collections import deque
import matplotlib.pyplot as plt

# Directions
directions = deque(["North", "East", "South", "West"])

# Duration each light stays green (in seconds)
green_light_duration = 5
num_cycles = 1

# Store timeline of green lights: List of (Direction, StartTime, Duration)
timeline = []

def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')

def show_traffic_lights(active, queue, seconds_left):
    print("=== Traffic Light Round Robin Simulation ===\n")
    print(f"Current Queue Order: {list(queue)}\n")
    for direction in directions:
        if direction == active:
            print(f"{direction}: 🟢 GREEN | Remaining Time: {seconds_left} sec")
        else:
            print(f"{direction}: 🟡 RED")

# Track total simulation time (for Gantt)
current_time = 0

for cycle in range(num_cycles * len(directions)):
    current_direction = directions[0]

    # Save the interval for Gantt chart
    timeline.append((current_direction, current_time, green_light_duration))

    for seconds_left in range(green_light_duration, 0, -1):
        clear_screen()
        print(f"Cycle {cycle + 1}: {current_direction}'s Turn")
        show_traffic_lights(current_direction, directions, seconds_left)
        time.sleep(1)

    current_time += green_light_duration
    directions.rotate(-1)

clear_screen()
print("Simulation Complete!\n")
```

```

# === GANTT CHART PLOTTING ===
fig, ax = plt.subplots(figsize=(10, 4))

colors = {
    "North": "tab:blue",
    "East": "tab:green",
    "South": "tab:orange",
    "West": "tab:red"
}

y_pos = {
    "North": 3,
    "East": 2,
    "South": 1,
    "West": 0
}

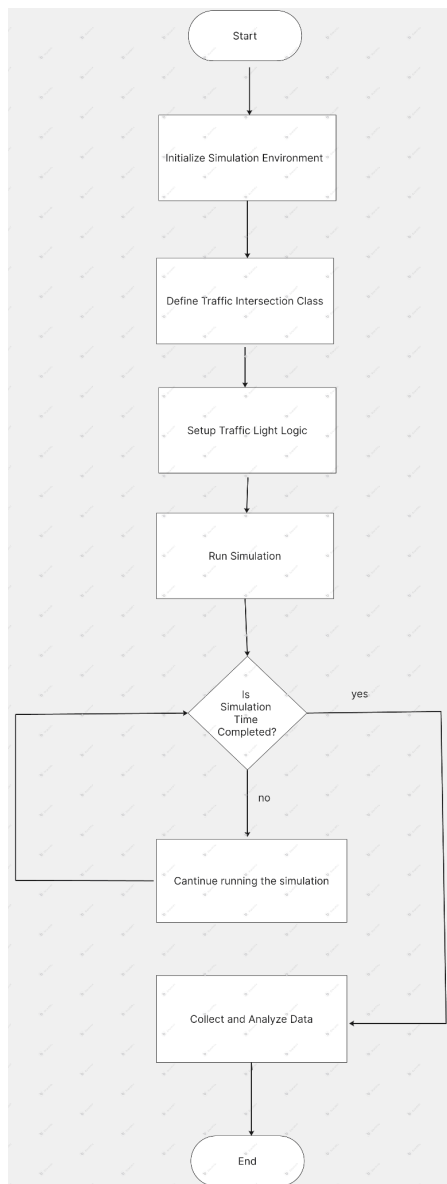
for direction, start_time, duration in timeline:
    ax.broken_barh([(start_time, duration)], (y_pos[direction], 0.9), facecolors=colors[direction])
    ax.text(start_time + duration / 2, y_pos[direction] + 0.45, f"{direction}", ha='center', va='center', color='white', fontsize=9)

ax.set_yticks([y_pos[d] + 0.45 for d in directions])
ax.set_yticklabels(directions)
ax.set_xlabel("Time (seconds)")
ax.set_title("Traffic Light Round Robin Schedule (Gantt Chart)")
ax.set_xlim(0, current_time)
ax.set_ylim(-0.5, 4)
ax.grid(True, axis='x', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```

5.1 System Flowchart



Steps in the Flowchart:

1. **Initialize Simulation Environment** : Setting up the Python environment and deque module.

2. **Define Traffic Intersection Class** : Creating the Traffic Intersection class with methods for traffic light control.kee

3. **Setup Traffic Light Logic** : define the number of lanes and the time for each light stays green.

4. **Run Simulation** : run the simulation by looping through each lane with Round Robin scheduling, and switching the traffic light direction

5. **Is Simulation Time finished?** : Checking if the total cycle simulation time has been reached.

- **No**: Continue running the simulation.
- **Yes**: Proceed to data collection and analysis.

6. **Collect and Analyze Data** : Record waiting times, traffic throughput, and other things.

6. Results & Discussion

Cycle 1: North's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['North', 'East', 'South', 'West']

North:  GREEN | Remaining Time: 5 sec

East:  RED

South:  RED

West:  RED

Cycle 1: North's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['North', 'East', 'South', 'West']

North:  GREEN | Remaining Time: 4 sec

East:  RED

South:  RED

West:  RED

Cycle 1: North's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['North', 'East', 'South', 'West']

North:  GREEN | Remaining Time: 3 sec

East:  RED

South:  RED

West:  RED

Cycle 1: North's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['North', 'East', 'South', 'West']

North:  GREEN | Remaining Time: 2 sec

East:  RED

South:  RED

West:  RED

Cycle 1: North's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['North', 'East', 'South', 'West']

North:  GREEN | Remaining Time: 1 sec

East:  RED

South:  RED

West:  RED

Cycle 2: East's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['East', 'South', 'West', 'North']

East:  GREEN | Remaining Time: 5 sec

South:  RED

West:  RED

North:  RED

Cycle 2: East's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['East', 'South', 'West', 'North']

East:  GREEN | Remaining Time: 4 sec

South:  RED

West:  RED

North:  RED

Cycle 2: East's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['East', 'South', 'West', 'North']

East:  GREEN | Remaining Time: 3 sec

South:  RED

West:  RED

North:  RED

Cycle 2: East's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['East', 'South', 'West', 'North']

East:  GREEN | Remaining Time: 2 sec

South:  RED

West:  RED

North:  RED

Cycle 2: East's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['East', 'South', 'West', 'North']

East:  GREEN | Remaining Time: 1 sec

South:  RED

West:  RED

North:  RED

Cycle 3: South's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['South', 'West', 'North', 'East']

South:  GREEN | Remaining Time: 5 sec

West:  RED

North:  RED

East:  RED

Cycle 3: South's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['South', 'West', 'North', 'East']

South:  GREEN | Remaining Time: 4 sec

West:  RED

North:  RED

East:  RED

Cycle 3: South's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['South', 'West', 'North', 'East']

South:  GREEN | Remaining Time: 3 sec

West:  RED

North:  RED

East:  RED

Cycle 3: South's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['South', 'West', 'North', 'East']

South:  GREEN | Remaining Time: 2 sec

West:  RED

North:  RED

East:  RED

Cycle 3: South's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['South', 'West', 'North', 'East']

South:  GREEN | Remaining Time: 1 sec

West:  RED

North:  RED

East:  RED

Cycle 4: West's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['West', 'North', 'East', 'South']

West:  GREEN | Remaining Time: 5 sec

North:  RED

East:  RED

South:  RED

Cycle 4: West's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['West', 'North', 'East', 'South']

West:  GREEN | Remaining Time: 4 sec

North:  RED

East:  RED

South:  RED

Cycle 4: West's Turn

=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['West', 'North', 'East', 'South']

West:  GREEN | Remaining Time: 3 sec

North:  RED

East:  RED

South:  RED

Cycle 4: West's Turn

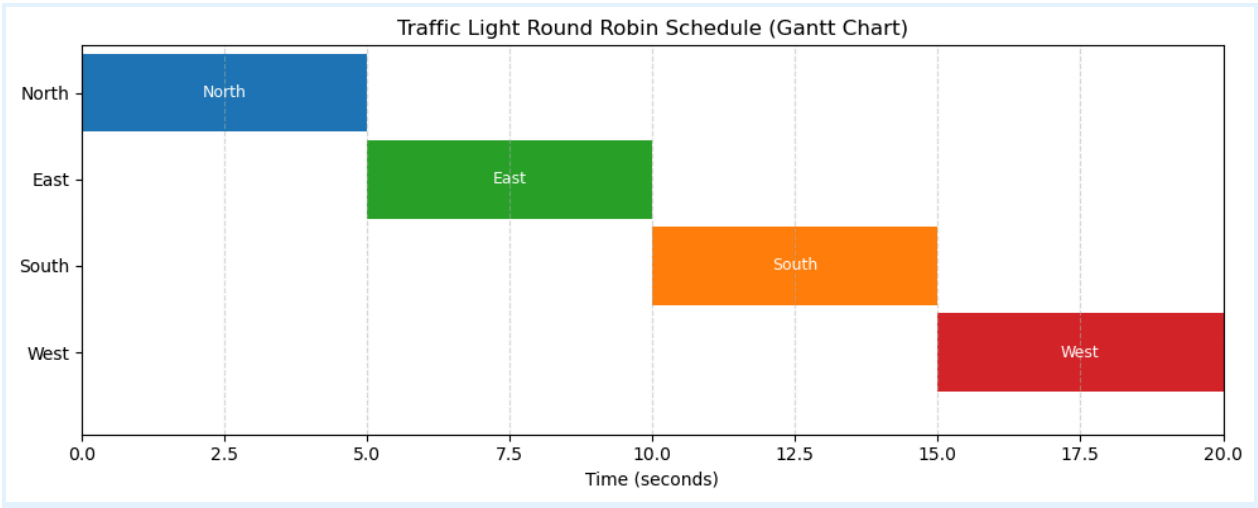
=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['West', 'North', 'East', 'South']

West: ● GREEN | Remaining Time: 2 sec
North: ● RED
East: ● RED
South: ● RED
Cycle 4: West's Turn
=== Traffic Light Round Robin Simulation ===

Current Queue Order: ['West', 'North', 'East', 'South']

West: ● GREEN | Remaining Time: 1 sec
North: ● RED
East: ● RED
South: ● RED
Simulation Complete!



6.1 Interface and Execution

The Simulation interface is a representation of the traffic light flow at a 4-way intersection. The simulation uses a python to run the logic and display how the traffic light change based on the round robin style.

The interface shows the status of each direction with the current simulation time. Here's how it works :

- Setup : Setting up the program for example how many roads there are and how long does the green light is last long
- Traffic Light Logic : The traffic light will turn green for one direction per time, then switch to the next direction.
- Data Tracking : The program track how long does the cars wait at each light
- Visualization : show which light are green or red and how time is passing through out the simulation.

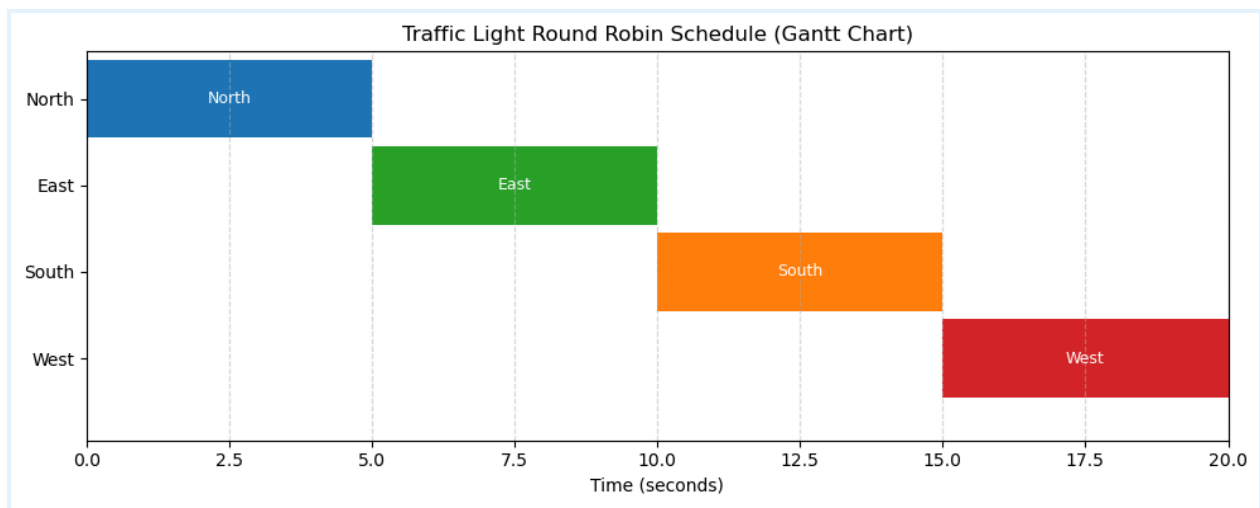
6.2 Output Analysis

The Simulation output helps us see how the Round Robin method works in controlling traffic lights. It shows useful information like :

- How long cars need to wait at each light
- How many cars can get through the intersection
- How efficient the traffic light flow is

Example output :

- North: ● GREEN | Remaining Time: 5 sec
- East: ● RED
- South: ● RED
- West: ● RED



6.3 Discussion

The simulation shows that using a Round Robin method for traffic lights can effectively distribute green light time between multiple directions. It helps traffic to move smoothly and reduces waiting times. These are the main things we can see from the results :

- **Balance Traffic Flow** : Each direction gets an equal amount of green light time, ensuring a balanced flow of traffic
- **Reduced Waiting Time** : The average waiting time for each direction has been minimized which helps to reduce traffic congestion.
- **Scalability** : The Round Robin method can be easily scaled to adapt intersection that have more lanes or heavier traffic

7. Future Work & Conclusion

7.1 Future Work

The simulation shows that the Round Robin method can be used for traffic lights effectively by distributing green light time fairly, but there are ways that can make it even better in the future :

- **Adaptive Scheduling** : adjusting the green light duration based on real time traffic condition
- **Use of Sensors** : Add a smart sensor to detect traffic in real time and adjust the duration of the light automatically
- **Advance visualization** : Create more advanced visualization tools to provide a clearer representation of traffic flow and where traffic jams happen

7.2 Conclusion

This project shows the application of the Round Robin Scheduling algorithm to traffic light control at a four way intersection. By distributing green light time among multiple directions evenly, the algorithm helps the traffic flow smoothly and minimizes waiting times. The simulation results show the potential of scheduling methods from operating systems that can solve real world challenges like traffic management by offering a useful insight for traffic engineers and city planners. In the future, the system will be focused on enhancing it by using adaptive scheduling, sensors to gather live data and advanced visualisation to further reduce traffic jams in busy city areas.

8. Acknowledgement

[Include acknowledgements here]

9. References

[Include references here]